

TrustLedger

A tamper-evident access-control audit log. Every grant and revoke is chained by hash, so any later edit to the history can be detected.

Links

- **Repository:** <https://github.com/Krishika711/TrustLedger>
- **Live demo:** <https://trust-ledger-eight.vercel.app/>

The problem

In a normal database, anyone with admin access can quietly edit or delete access-log rows, and there is no way to prove the trail wasn't altered.

The solution

Every grant and revoke is written to a hash-chained ledger. Each event's hash includes the previous event's hash:

```
event_hash = SHA256(prev_hash + actor_id + resource_id + action + created_at)
```

The first event uses a `prev_hash` of 64 zeros. If any row is edited after the fact, recomputing the chain shows exactly which events no longer match. This is a plain hash chain in PostgreSQL. It is **not** a blockchain deployment (no smart contracts, wallets, or gas).

Features

- Admin dashboard: sign in, view the current access matrix, grant and revoke access, see ledger receipts
- Audit dashboard: verified/tampered status, full event list with actor, resource, date/time, and per-event tamper flags
- Admins can mark flagged events as reviewed (stored in the browser only; events are never hidden)
- Admin-only grant/revoke enforced by an RBAC check on the server
- Serialised ledger writes so concurrent requests can't fork the chain

Tech stack

Next.js 14 + React 18 (client) · Node.js + Express 4 (API) · PostgreSQL via `pg` (hosted on Supabase) · Node `crypto` for SHA-256. See [docs/TECH_STACK.md](#).

Repository layout

```
server/
  src/index.js           Express app, mounts all routes
  src/routes/            auth, access, ledger, audit
  src/utils/hashChain.js hashing, ledger insert, chain verification
  src/utils/rbac.js      canAccess() permission check
  src/db/                pool.js, schema.sql, seed.sql
  test-rbac.js           manual RBAC test script
client/
  pages/                 index (landing), admin, audit
  components/admin/      Header, LoginPanel, AccessMatrix, Composer, Receipts
  lib/                   api, adminApi, session, directory
```

Getting started

Prerequisites: Node.js 18+, a PostgreSQL database (Supabase or local), and `psql` if you want to use `npm run migrate`.

Server

```
cd server
cp .env.example .env      # set DATABASE_URL and PORT
npm install
npm run migrate           # creates tables (or run src/db/schema.sql in your DB client)
psql "$DATABASE_URL" -f src/db/seed.sql # optional: demo users and resources
npm run dev               # http://localhost:4000
```

Client

```
cd client
cp .env.example .env      # NEXT_PUBLIC_API_BASE=http://localhost:4000
npm install
npm run dev               # http://localhost:3000
```

Never commit a real `DATABASE_URL`; keep credentials in `.env` only.

Demo flow

1. Sign in on `/admin` as an admin user (the seed data includes `Alice`, role `admin`).
2. Grant and revoke a permission. Each action writes to `permissions` and appends to `ledger_events`.
3. Open `/audit`: the status card shows the chain is verified.
4. Tamper directly in the database, for example:

```
UPDATE ledger_events SET action = 'grant:read' WHERE id = 3;
```

5. Click **Re-check** on `/audit`: the status turns to tampered and event #3 is flagged.

Documentation

- [Architecture](#)
- [Tech stack](#)
- [API reference](#)

Team

Member	Area
Saanvi	Backend API (auth, access routes)
Krishika	Hash chain, audit verification, integration
Basant	Database schema and RBAC
Shickey	Admin dashboard
Tejas	Audit dashboard
Hritika	Seed data, QA, demo

Known limitations

- Login is demo-only: it matches a user by name and returns a fixed token. The API does not yet verify that token, and grant/revoke trust the `granted_by` / `revoked_by` value in the request body.
- Tamper detection catches edited rows and broken links. It cannot detect deletion of the most recent rows, and someone with full database write access who recomputes every later hash could forge a valid chain. Publishing the latest hash somewhere outside the database would close this gap.
- Grant/revoke update `permissions` and `ledger_events` in two separate queries, not one transaction.
- The write queue serialises ledger inserts within one server process only.